ChatGPT

Agentic Prospecting Widget UI (React + TailwindCSS + Framer Motion)

We build an Electron-based desktop widget with a **glassmorphic** UI (frosted-glass panels, soft shadows, thin borders [1]) and smooth animations via Framer Motion. We use React + TailwindCSS as the framework. Tailwind's utility-first classes handle styling while Framer Motion provides declarative animation props [2]. Our code simulates Claude Opus 4.1's agent flow on the client (future integration with Claude's API is planned) – Opus 4.1 is Anthropic's latest LLM that "strengthens coding reliability… and improves reasoning" across long interactions [3].

- **Glassmorphic Design:** UI panels (cards, modals) use semi-transparent backgrounds with CSS `backdrop-blur` and soft shadows to achieve a frosted-glass effect [1]. For example, a class like `bg-white/20 backdrop-blur-md shadow-lg border border-white/25` creates a translucent card.
- **React + TailwindCSS:** We organize the UI into modular React components for each screen, using Tailwind utility classes for layout and styling (grids, flex, padding, colors, etc.).
- **Framer Motion Animations:** Screen and element transitions (fade-in, slide, progress animations) use `<motion.div>` and AnimatePresence. As one guide notes, combining Tailwind with Framer Motion "enables you to craft … interactions – all while keeping your code clean and responsive" [2].
- **State & Simulation:** All logic is client-side. We use React hooks (`useState`, `useEffect`) to manage state and simulate asynchronous steps. For example:
- *Validation:* A stub `validateURL(url)` returns a Promise (after a timeout) indicating if a URL is valid or broken (we can use a simple regex).
- *Pre-scan:* A stub `runPreScan(urls)` returns mock data for each company (e.g. quality score 0–100, senior role count, estimated time).
- *Enrichment:* A stub `enrichData(companies, onProgress)` simulates scanning each name with an increasing progress percentage and confidence scores.
- *Prioritization:* A stub `agentPrioritize(data)` returns an array of JSON objects with `priorityScore`, `reason`, and timing suggestions for each prospect.
- *Errors:* We simulate occasional errors by randomly marking some items as failed (showing an inline error state with an option like "Resolve later").

Each UI screen corresponds to a phase of the agentic flow. Below we describe each screen's design and show key React components. The code omits full backend hookup and uses placeholder data.

Screen 1: Workspace Home

This is the landing page showing saved campaigns and recent prospecting results. Key features:

- **Campaign Cards:** Display each campaign's name, type, timestamp, and result count. Use a translucent "glass" card style with `backdrop-blur` and shadow [1].
- **Status & Resume:** Each card shows a status indicator (e.g. a colored dot or icon) and a "Resume" button to continue a past run.
- **New Campaign:** A button to start a new campaign run.

We implement this as a `WorkspaceHome` component that takes a `campaigns` array prop. Each card uses Framer Motion for a subtle fade-in. For brevity, icons here are represented by text or emoji placeholders.

```
function WorkspaceHome({ campaigns, onResume, onNew }) {
  return (
    <div className="p-6 space-y-4">
      <h2 className="text-2xl font-bold">Saved Campaigns</h2>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        {campaigns.map((c) => (
          <motion.div
            key={c.id}
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            className="bg-white/10 backdrop-blur-md shadow-lg border border-
white/25 rounded-xl p-4"
          >
            <h3 className="font-semibold text-lg">{c.name}</h3>
            <p className="text-sm text-gray-200">{c.type} • {new
Date(c.timestamp).toLocaleString()}</p>
            <p>{c.resultsCount} results</p>
            <div className="flex items-center mt-2 space-x-2">
              {/* Status icon (green=done, orange=running, etc.) */}
              <span className={`h-3 w-3 rounded-full ${c.status ===
'running' ? 'bg-yellow-400' : 'bg-green-400'}`}></span>
              <button
                onClick={() => onResume(c.id)}
                className="px-3 py-1 bg-blue-500 text-white rounded-md text-
sm"
              >
                Resume
              </button>
            </div>
          </motion.div>
        ))}
      </div>
      <button
        onClick={onNew}
        className="mt-4 px-6 py-3 bg-green-500 text-white rounded-md shadow-
md hover:bg-green-600"
      >
        + New Campaign
      </button>
    </div>
  );
}
```

Screen 2: Smart Input & Validation

In the **Smart Setup** phase the user inputs one or more company URLs and names the campaign. Features include:

- **URL Entry:** A list of URL input fields. The user can add more URLs with a "+ Add" button.
- **Real-time Validation:** As the user types each URL, we call the `validateURL` stub after a debounce (e.g. 300ms) to simulate checking reachability. We then show an icon: a green check (✓) for valid or a red cross (✗) for broken/unsupported.
- **Campaign Name:** An input for naming the campaign.
- **Pre-scan Toggle:** A checkbox or switch labeled "Pre-scan" (default on) that lets the user preview data quality before full enrichment.
- **Next Action:** A button to proceed. Its label changes based on the toggle ("Run Pre-Scan" vs "Skip to Enrichment").

Example component with Tailwind styling and simple state hooks:

```
function SmartInput({ onNext }) {
  const [urls, setUrls] = useState([""]);
  const [statuses, setStatuses] = useState([]);
  const [name, setName] = useState("");
  const [prescan, setPrescan] = useState(true);

  useEffect(() => {
    // Debounced validation of each URL
    const handle = setTimeout(() => {
      const checks = urls.map((u) => validateURL(u).then(res => res.status));
      Promise.all(checks).then((results) => setStatuses(results));
    }, 300);
    return () => clearTimeout(handle);
  }, [urls]);

  return (
    <div className="p-6 space-y-4">
      <h2 className="text-xl font-bold">Enter Company URLs</h2>
      {urls.map((u, i) => (
        <div key={i} className="flex space-x-2 items-center">
          <input
            type="text"
            value={u}
            onChange={(e) => {
              const list = [...urls]; list[i] = e.target.value;
              setUrls(list);
            }}
            placeholder="https://example.com"
            className="flex-1 p-2 rounded bg-white/20 backdrop-blur-sm text-
white"
          />
          <span className="w-6">
            {statuses[i] === "Valid" && <span className="text-green-400">✓</
```

```
span>}
            {statuses[i] === "Broken" && <span className="text-red-400">✗</
span>}
            {!statuses[i] && <span className="text-gray-400">…</span>}
          </span>
        </div>
      ))}
      <button
        onClick={() => setUrls([...urls, ""])}
        className="text-sm text-blue-300 hover:underline"
      >
        + Add another URL
      </button>
      <input
        type="text"
        value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Campaign Name"
        className="w-full mt-4 p-2 rounded bg-white/20 backdrop-blur-sm text-
white"
      />
      <label className="flex items-center mt-2">
        <input
          type="checkbox"
          checked={prescan}
          onChange={e => setPrescan(e.target.checked)}
          className="mr-2"
        />
        <span>Pre-scan before enrichment</span>
      </label>
      <button
        onClick={() => onNext({ urls, name, prescan })}
        className="mt-4 px-6 py-2 bg-blue-500 text-white rounded-md hover:bg-
blue-600"
      >
        {prescan ? "Run Pre-Scan" : "Skip to Enrichment"}
      </button>
    </div>
  );
}
```

In this code, `validateURL` is a simulated async function (stub) that might simply resolve as valid if the URL starts with `http`. The real implementation would check URL format and maybe ping the site.

Screen 3: Pre-Scan Decision Gate

After running the pre-scan (if enabled), we show a **summary** of findings. This allows the human to decide the next step. Key elements:

- **Summary Info:** e.g. “Found 42 names across 3 companies, including 10 senior roles.”

- **Table of Companies:** Columns for company name, quality/confidence score, estimated enrichment time, and a scope selector (All, Senior-only, Manual).
- **Proceed Button:** A "Start Enrichment" button to confirm and move on.

We model this with a `PreScanGate` component. (We assume `runPreScan` returned data like `{ company, quality, seniorCount, estimatedTime }`.)

```
function PreScanGate({ data, onProceed }) {
  const totalNames = data.reduce((sum, c) => sum + c.seniorCount, 0);
  return (
    <div className="p-6 space-y-4">
      <h2 className="text-xl font-bold">Pre-Scan Results</h2>
      <p>Found {totalNames} names across {data.length} companies (incl.
senior roles).</p>
      <table className="min-w-full table-auto text-white">
        <thead>
          <tr>
            <th className="text-left">Company</th>
            <th className="text-left">Quality (%)</th>
            <th>Est. Time</th>
            <th>Scope</th>
          </tr>
        </thead>
        <tbody>
          {data.map((c, i) => (
            <tr key={i} className="bg-white/10 backdrop-blur-sm">
              <td>{c.company}</td>
              <td>{c.quality}</td>
              <td>{c.estimatedTime}</td>
              <td>
                <select defaultValue="all" className="bg-white/20 p-1 text-
black rounded">
                  <option value="all">All</option>
                  <option value="senior">Senior Only</option>
                  <option value="manual">Manual</option>
                </select>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
      <button
        onClick={onProceed}
        className="mt-4 px-6 py-2 bg-green-500 text-white rounded-md
hover:bg-green-600"
      >
        Start Enrichment
      </button>
    </div>
```

```
  );
}
```

The table rows use a translucent background to continue the glass effect. The “Scope” column uses a select dropdown for the user to filter (though in this demo code it does not affect logic).

Screen 4: Error-Resilient Progress

During **enrichment**, we show a live progress view that is resilient to errors. Features:

- **Progress Bar:** A full-width bar indicating overall progress (0–100%). We use a `<motion.div>` to animate the width smoothly.
- **Item List:** As each name is processed, we append a row showing the name (or role), its confidence score, and status. If an error occurs, we display an inline message or color (e.g. red text “Error”) and offer a “Resolve later” button.
- **Confidence Scores:** Each item row shows an AI-generated confidence (simulated here as a percentage).
- **Resume/Resolve:** An option to skip ambiguous entries.

Example `EnrichmentProgress` component (calls a stub `enrichData` that invokes a callback per item):

```
function EnrichmentProgress({ companies, onComplete }) {
  const [items, setItems] = useState([]);
  const [progress, setProgress] = useState(0);

  useEffect(() => {
    if (companies.length === 0) return;
    const total = companies.length;
    // Simulate asynchronous enrichment with callbacks
    enrichData(companies.map(c => c.company), (company, pct, item) => {
      setProgress(pct);
      setItems((prev) => [...prev, item]);
      if (pct === 100) {
        onComplete(prev => {}, );  // placeholder, trigger move to next
screen
      }
    });
  }, []);

  return (
    <div className="p-6 space-y-4">
      <h2 className="text-xl font-bold">Enrichment Progress</h2>
      <div className="w-full bg-gray-700 rounded-full h-2">
        <motion.div
          className="bg-blue-500 h-2 rounded-full"
          style={{ width: `${progress}%` }}
          initial={{ width: 0 }}
          animate={{ width: `${progress}%` }}
          transition={{ ease: "linear", duration: 0.5 }}
```

```
        />
      </div>
      <div className="space-y-2 mt-4">
        {items.map((itm, i) => (
          <div key={i} className="flex justify-between items-center bg-white/
10 p-2 rounded">
            <span>{itm.name} {itm.error && <span className="text-
red-400">(Error)</span>}</span>
            <span className="text-sm">{itm.confidence || '--'}%</span>
            {itm.error ? (
              <button className="text-yellow-300 text-sm">Resolve later</
button>
            ) : (
              <span className="text-green-400 text-sm">OK</span>
            )}
          </div>
        ))}
      </div>
    </div>
  );
}
```

In this stub, `enrichData` would take the list of companies and periodically call the callback with the latest progress percentage and a mock `item` (e.g. `{name: "...", confidence: "75"}` or `{error: true}`). On completion (100%), it calls `onComplete` to move to results.

Screen 5: AI-Powered Results

Once enrichment is done, we display an **AI analysis** of the prospects. This includes:

- **Prioritization Recommendations:** For each prospect, we show an AI-generated priority score and a reason (both simulated). For example: "John Doe – Score 85: *Reason: High seniority and recent company funding round*".
- **Timing Suggestions:** A line like "Best time to reach out: Q2 2026" (simulated).
- **Bulk Actions:** Buttons to export data, assign leads to a sales pipeline, or notify an advisor.
- **Animations:** We can animate each item's entrance with Framer Motion (sliding in with fade).

Example `AiResults` component:

```
function AiResults({ prospects }) {
  return (
    <div className="p-6 space-y-4">
      <h2 className="text-xl font-bold">AI-Powered Prioritization</h2>
      {prospects.map((p, i) => (
        <motion.div
          key={i}
          initial={{ opacity: 0, x: -20 }}
          animate={{ opacity: 1, x: 0 }}
          transition={{ delay: i * 0.1 }}
          className="bg-white/10 backdrop-blur-md p-4 rounded-md"
```

```
        >
          <p>
            <strong>{p.name}</strong> &mdash; Score: {p.priorityScore}
          </p>
          <p className="text-sm text-gray-300">{p.reason}</p>
          <p className="text-sm text-gray-300">Best time: {p.timing}</p>
        </motion.div>
      ))}
      <div className="mt-6 flex space-x-2">
        <button className="px-4 py-2 bg-gray-500 text-white rounded hover:bg-
gray-600">Export</button>
        <button className="px-4 py-2 bg-blue-500 text-white rounded hover:bg-
blue-600">Assign to Pipeline</button>
        <button className="px-4 py-2 bg-green-500 text-white rounded
hover:bg-green-600">Notify Advisor</button>
      </div>
    </div>
  );
}
```

Here `prospects` is an array of objects like `{ name, priorityScore, reason, timing }` returned by our `agentPrioritize` stub.

Screen 6: Enhanced Prospect Details

Clicking on a prospect opens a detail view. It shows full information and tools for contact:

- **Detail Breakdown:** Full name, contact info (email, title), confidence scores, source links.
- **Reasoning:** The AI's rationale for prioritization (expanded from Screen 5).
- **Inline Contact Actions:** Buttons like "Copy Email" or "Connect".
- **Glass Style:** Keep the same translucent panel look.

Example `ProspectDetails` component:

```
function ProspectDetails({ prospect }) {
  return (
    <div className="p-6 space-y-4">
      <h2 className="text-xl font-bold">{prospect.name}</h2>
      <p><strong>Email:</strong> <span className="text-
blue-400">{prospect.email}</span></p>
      <p><strong>Confidence:</strong> {prospect.confidence || 'N/A'}%</p>
      <p><strong>Source:</strong> <a href={prospect.source}
className="underline">Company Site</a></p>
      <p><strong>AI Reason:</strong> {prospect.reason}</p>
      <div className="flex space-x-2 mt-4">
        <button className="px-4 py-2 bg-gray-500 text-white rounded hover:bg-
gray-600">Copy Email</button>
        <button className="px-4 py-2 bg-blue-500 text-white rounded hover:bg-
blue-600">Connect</button>
      </div>
```

```
    </div>
  );
}
```

In a full app, `ProspectDetails` could be rendered in a modal or a routed screen. Here it's a standalone component.

Client-Side Simulation Stubs

All asynchronous operations use client-side stubs. For example:

```
// Simulate URL validation
function validateURL(url) {
  return new Promise(resolve => {
    setTimeout(() => {
      const isValid = /^https?:\/\//.test(url);
      resolve({ url, status: isValid ? "Valid" : "Broken" });
    }, 500);
  });
}

// Simulate pre-scan of websites
function runPreScan(urls) {
  return new Promise(resolve => {
    setTimeout(() => {
      const results = urls.map(url => {
        const name = new URL(url).hostname.replace('www.', '');
        return {
          company: name,
          quality: Math.floor(Math.random() * 50) + 50,
          seniorCount: Math.floor(Math.random() * 5) + 1,
          estimatedTime: `${Math.floor(Math.random() * 3) + 1}m`
        };
      });
      resolve(results);
    }, 1000);
  });
}

// Simulate enrichment (calls onProgress callback per company)
function enrichData(companies, onProgress) {
  companies.forEach((company, i) => {
    const delay = (i + 1) * 1000;
    setTimeout(() => {
      const item = {
        name: `Lead from ${company}`,
        confidence: Math.floor(Math.random() * 50) + 50
      };
      const pct = Math.round(((i + 1) / companies.length) * 100);
      onProgress(company, pct, item);
```

```
    }, delay);
  });
}

// Simulate AI prioritization
function agentPrioritize(prospects) {
  return prospects.map(p => ({
    ...p,
    priorityScore: Math.floor(Math.random() * 50) + 50,
    reason: "Based on recent funding and senior role.",
    timing: "Next quarter"
  }));
}
```

These stubs introduce artificial delays and mock data so the UI behaves realistically (with loading states, progress, etc.). Error conditions can be simulated by randomly injecting `item.error = true` for some entries, causing the UI to show a failure and offer "Resolve later".

Putting It All Together

In `App.js`, we tie these components and stubs with React state. For example:

```
function App() {
  const [screen, setScreen] = useState("home");
  const [campaigns, setCampaigns] = useState([]);
  const [inputData, setInputData] = useState(null);
  const [preScanData, setPreScanData] = useState([]);
  const [finalProspects, setFinalProspects] = useState([]);
  const [selectedProspect, setSelectedProspect] = useState(null);

  const handleNew = () => setScreen("input");
  const handleResume = (id) => { /* load campaign state */ };

  const handleInputNext = ({ urls, name, prescan }) => {
    setCampaigns([{id:1, name, type:"Custom", timestamp: Date.now(),
resultsCount:0, status:"running"}, ...campaigns]);
    if (prescan) {
      runPreScan(urls).then(data => {
        setPreScanData(data);
        setScreen("prescan");
      });
    } else {
      setScreen("progress");
      setPreScanData([]);
    }
  };

  const handleProceed = () => setScreen("progress");
  const handleEnrichComplete = (prospects) => {
```

```
    const results = agentPrioritize(prospects);
    setFinalProspects(results);
    setScreen("results");
  };

  return (
    <div className="min-h-screen bg-gray-800 text-white">
      {screen === "home" && (
        <WorkspaceHome
          campaigns={campaigns}
          onResume={handleResume}
          onNew={handleNew}
        />
      )}
      {screen === "input" && (
        <SmartInput onNext={handleInputNext} />
      )}
      {screen === "prescan" && (
        <PreScanGate data={preScanData} onProceed={handleProceed} />
      )}
      {screen === "progress" && (
        <EnrichmentProgress
          companies={preScanData.map(c => c.company)}
          onComplete={handleEnrichComplete}
        />
      )}
      {screen === "results" && (
        <AiResults prospects={finalProspects} />
      )}
      {screen === "detail" && selectedProspect && (
        <ProspectDetails prospect={selectedProspect} />
      )}
    </div>
  );
}
```

This central `App` component manages which screen is shown and passes data/handlers down. All logic (validation, scanning, enrichment, AI reasoning) happens in these simulated client-side functions, so no backend API is required. In a production version, the stubs would be replaced by real calls to the Claude Opus 4.1 API.

In summary, the UI uses a modern glassmorphic aesthetic [1] with responsive animations via Framer Motion [2]. Each of the four major agentic phases is mapped to a screen as described, and local React state (and simulated async functions) drive the flow. This frontend is thus fully ready to integrate the real Claude model in the future, while providing the user with a polished, interactive prospecting assistant interface.

Sources: Design and animation practices [1] [2] and Claude 4.1 capabilities [3].

---

1 What is Glassmorphism? UI Design Trend 2025
https://www.designstudiouiux.com/blog/what-is-glassmorphism-ui-trend/

2 Animating React Components with Tailwind and Framer Motion | Hoverify
https://tryhoverify.com/blog/animating-react-components-with-tailwind-and-framer-motion/

3 Anthropic’s Claude Opus 4.1 Improves Refactoring and Safety, Scores 74.5% SWE-bench Verified - InfoQ
https://www.infoq.com/news/2025/08/anthropic-claude-opus-4-1/